

Universitatea Națională de Știință și Tehnologie
POLITEHNICA din București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect Microcontrolerul ESP32 CAM
„Echipament mobil cu monitorizare video și
control prin interfață web”

Petrescu Daniel-Adrian

I. PREZENTAREA PROIECTULUI:

Proiectul constă în realizarea unui robot mobil cu monitorizare video și control prin interfață web. Sistemul utilizează camere ESP32-CAM pentru captarea și transmiterea imaginilor video în timp real. Camerele sunt echipate cu senzori PIR pentru detectarea mișcării.

La detectarea unei mișcări, sistemul realizează automat o fotografie și transmite o notificare prin Telegram, împreună cu imaginea capturată și numărul camerei care a detectat mișcarea.

Imaginile video provenite de la cele patru camere pot fi urmărite simultan prin intermediul unei interfețe web, ceea ce permite monitorizarea spațiului supravegheat într-un mod simplu și centralizat.

Scopul proiectului este de a realiza un sistem accesibil și ușor de utilizat, care combină mobilitatea robotului, monitorizarea video în timp real și detectarea automată a mișcării.

II. COMPONENTE NECESARE:

În vederea elaborării proiectului, vom avea nevoie de următoarele componente:

- ✓ 4x Microcontrolere ESP32 CAM
- ✓ 4x Senzori de mișcare PIR
- ✓ 4x Surse de alimentare a circuitului (Acumulatori 18650)
- ✓ 4x Limitatoare de tensiune LM2596
- ✓ 8x Perfoboard-uri pentru realizarea camerei
- ✓ Fludor
- ✓ Stație de lipit

III. CONEXIUNILE COMPONENTELOR:

1. Senzorul PIR

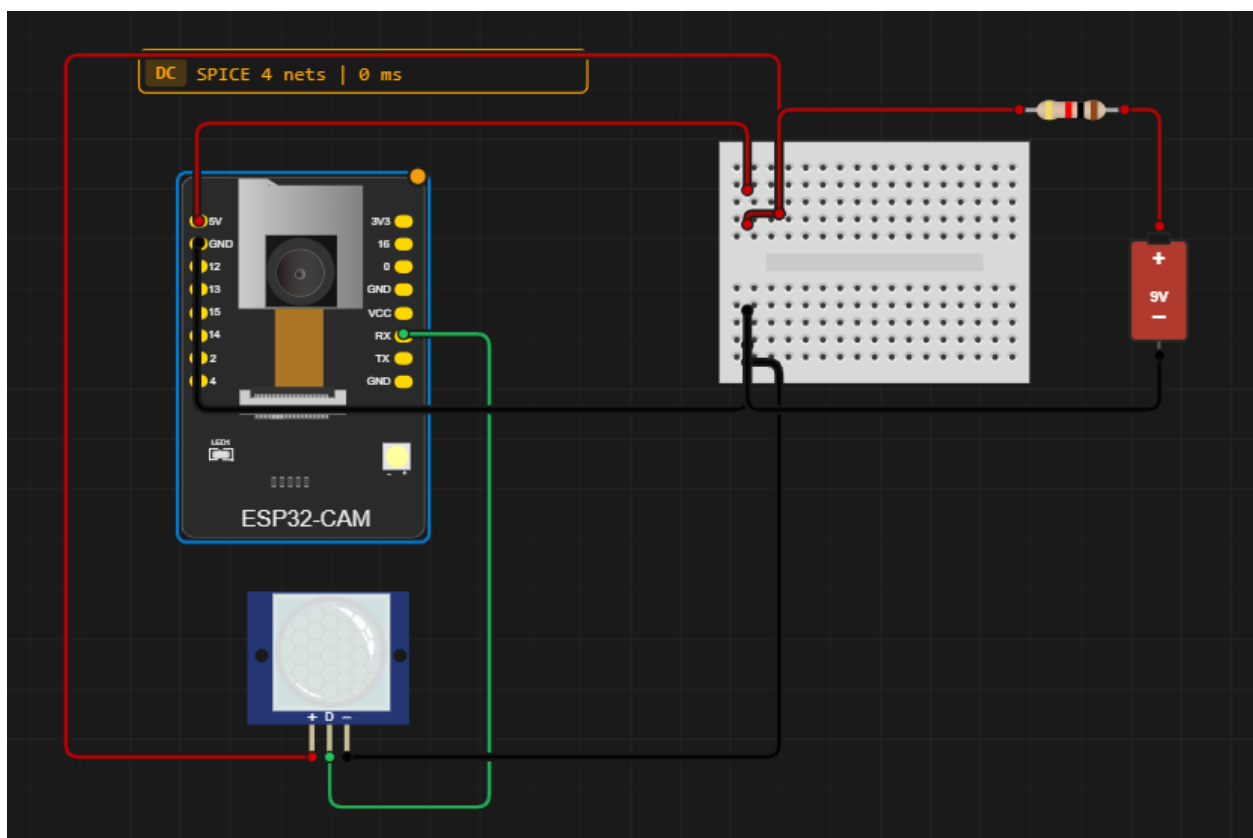
Componentă	ESP32-CAM
VCC – PIR	5V
GND – PIR	GND
OUT – PIR	GPIO3 (U0R)

2. Alimentarea:

Componentă	ESP32-CAM
Alimentare	5V
Masă	GND

Alimentarea si masa vor fi comune pentru cele 2 componente. Se vor lipi pe perfoboard co fludor astfel incat sa se respecte schema atasata mai jos.

IV. SCHEMA PROIECTULUI:



Specificatie: In schema realizata mai sus rezistenta reprezinta modulul LM2596 care inainte sa fie conectat efectiv la microcontroller si senzorul PIR trebuie reglat potentiometrul stabilizatorului astfel incat sa nu depaseasca valoarea de 5V !

V. EXPLICATIE PROIECT:

La pornirea sistemului sunt inițializate modulele ESP32-CAM, conexiunea la rețeaua Wi-Fi și senzorii PIR. Fiecare dintre cele patru camere are asociat un senzor PIR, utilizat pentru detectarea mișcării în zona monitorizată.

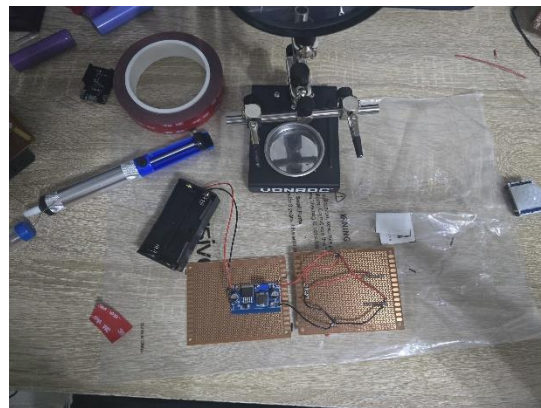
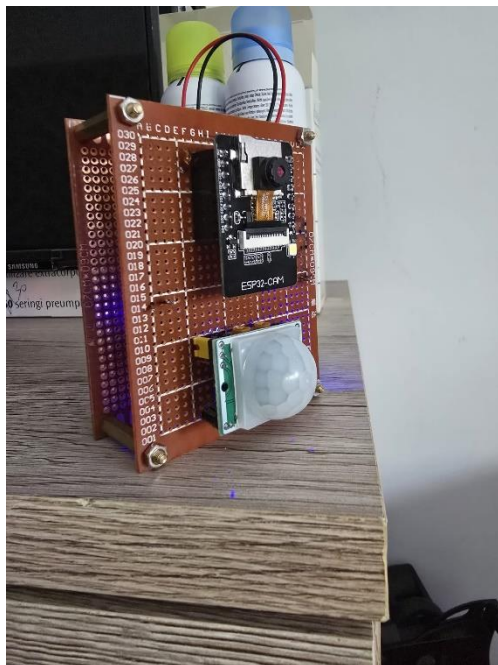
În timpul funcționării, camerele transmit imaginile video în timp real către interfața web. Cele patru fluxuri video pot fi vizualizate simultan, într-o interfață centralizată, ceea ce permite monitorizarea mai multor zone în același timp.

Senzorii PIR verifică permanent existența unei mișcări. Atunci când este detectată o mișcare, ESP32-CAM realizează automat o fotografie. Imaginea capturată este transmisă prin intermediul conexiunii Wi-Fi către un bot Telegram, împreună cu informația privind camera care a detectat mișcarea.

Pentru evitarea transmiterii repetate a unor notificări pentru aceeași mișcare, sistemul utilizează un interval de așteptare între două detectări succesive. După terminarea acestui interval, sistemul poate detecta și procesa o nouă mișcare.

Interfața web permite urmărirea simultană a celor patru camere, fiecare cameră fiind afișată separat și identificată prin numărul său. Astfel, sistemul combină monitorizarea video în timp real cu detectarea automată a mișcării și transmiterea notificărilor la distanță.

VI. POZE CU PROIECTUL:



VII. COD ARDUINO:

```
#include "esp_camera.h"
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>

// =====
// CAMERA X
// ESP32-CAM AI-THINKER
// PIR + TELEGRAM + WEB INTERFACE + LIVE STREAM
// =====

// =====
// WIFI
// =====

const char* ssid = "NUMELE RETELEI TALE";
const char* password = "NUMELE PAROLEI TALE";

// =====
// TELEGRAM
// =====

const char* BOT_TOKEN = "BOT TOKEN-UL TAU";
const char* CHAT_ID = "CHAT ID-UL TAU";

// =====
// PIR
// =====

// PIR OUT conectat la U0R / GPIO3
#define PIR_PIN 3

bool motionActive = false;

unsigned long lastMotionTime = 0;
```

```

// Timp minim intre doua alerte
const unsigned long MOTION_COOLDOWN = 10000;

// =====
// CAMERA AI-THINKER ESP32-CAM
// =====

#define PWDN_GPIO_NUM  32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM  0

#define SIOD_GPIO_NUM  26
#define SIOC_GPIO_NUM  27

#define Y9_GPIO_NUM  35
#define Y8_GPIO_NUM  34
#define Y7_GPIO_NUM  39
#define Y6_GPIO_NUM  36
#define Y5_GPIO_NUM  21
#define Y4_GPIO_NUM  19
#define Y3_GPIO_NUM  18
#define Y2_GPIO_NUM   5

#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM  23
#define PCLK_GPIO_NUM  22

// =====
// WEB SERVER
// =====

WebServer server(80);

// =====

```

```

// INIZIALIZARE CAMERA
// =====

bool initCamera() {

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;

    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;

    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;

    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;

    config.xclk_freq_hz = 20000000;

    config.pixel_format = PIXFORMAT_JPEG;

    // =====
    // SETARE IMAGINE
    // =====

```

```

if (psramFound()) {

    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;

    Serial.println("PSRAM detectat - VGA");

} else {

    config.frame_size = FRAMESIZE_QVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;

    Serial.println("Fara PSRAM - QVGA");
}

// =====
// INITIALIZARE
// =====

esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {

    Serial.printf(
        "Eroare initializare camera: 0x%x\n",
        err
    );

    return false;
}

Serial.println(
    "Camera initializata cu succes."

```



```

);

return true;
}

// =====
// CONECTARE WIFI
// =====

void connectWiFi() {

    Serial.println();
    Serial.println("Conectare la Wi-Fi...");

    WiFi.mode(WIFI_STA);

    // CORECTAT:
    // folosim ssid si password, exact cum sunt definite sus
    WiFi.begin(
        ssid,
        password
    );

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);

        Serial.print(".");
    }

    Serial.println();

    Serial.println(
        "Wi-Fi conectat!"
    );
}

```

```
Serial.print(  
  "IP Camera X: "  
);
```

```
Serial.println(  
  WiFi.localIP()  
);  
}
```

```
// =====  
// PAGINA WEB  
// =====
```

```
void handleRoot() {
```

```
  String html = R"rawliteral(  
  
<!DOCTYPE html>  
  
<html lang="ro">  
  
<head>  
  
<meta charset="UTF-8">  
  
<meta name="viewport"  
  content="width=device-width, initial-scale=1.0">  
  
<title>Smart Security Camera - Camera X</title>  
  
<style>  
  
body {  
  
  margin: 0;
```

```
padding: 20px;
```

```
background: #111;
```

```
color: white;
```

```
font-family: Arial, sans-serif;
```

```
text-align: center;
```

```
}
```

```
h1 {
```

```
margin-bottom: 5px;
```

```
}
```

```
.status {
```

```
margin: 15px;
```

```
font-size: 18px;
```

```
}
```

```
.camera {
```

```
max-width: 900px;
```

```
margin: auto;
```

```
}
```

```
#stream {
```

```
width: 100%;
```

```
max-width: 800px;
```

```
border-radius: 10px;
```

```
border: 2px solid #444;
```

```
}
```

```
.info {
```

```
margin-top: 15px;
```

```
color: #bbb;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="camera">
```

```
<h1>
```

```
 Camera X
```

```
</h1>
```

```
<div class="status">
```

```
□ Camera online
```

```
</div>
```

```

```

```
<div class="info">
```

Smart Security Camera

```
<br><br>
```

IP:)rawliteral";

```
html += WiFi.localIP().toString();
```

```
html += R"rawliteral(
```

```
<br>
```

PIR: GPIO3 / U0R

```
<br>
```

Telegram: ACTIV

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
)rawliteral";
```

```
server.send(  
    200,  
    "text/html; charset=UTF-8",  
    html  
);  
}
```

```
// =====  
// LIVE STREAM  
// =====
```

```
void handleStream() {
```

```
    WiFiClient client = server.client();
```

```
    client.println(  
        "HTTP/1.1 200 OK"  
    );
```

```
    client.println(  
        "Content-Type: multipart/x-mixed-replace; boundary=frame"  
    );
```

```
    client.println(  
        "Cache-Control: no-cache"  
    );
```

```
    client.println(  
        "Access-Control-Allow-Origin: *"  
    );
```

```
    client.println(  
        "
```

```
        "Connection: close"
    );

    client.println();

    // =====
    // TRIMITERE CADRE
    // =====

    while (client.connected()) {

        camera_fb_t* fb =
            esp_camera_fb_get();

        if (!fb) {

            Serial.println(
                "Eroare captura stream!"
            );

            break;
        }

        client.println(
            "--frame"
        );

        client.println(
            "Content-Type: image/jpeg"
        );

        client.println(
            "Content-Length: " +
            String(fb->len)
        );
    }
}
```

```
client.println();
```

```
client.write(  
    fb->buf,  
    fb->len  
);
```

```
client.println();
```

```
esp_camera_fb_return(fb);
```

```
    delay(40);  
}
```

```
client.stop();  
}
```

```
// =====  
// TELEGRAM - TRIMITERE FOTO  
// =====
```

```
bool sendPhotoTelegram(  
    camera_fb_t* fb  
) {
```

```
    WiFiClientSecure client;
```

```
    // Pentru prototip  
    client.setInsecure();
```



```
const char* host =  
    "api.telegram.org";
```

```
Serial.println(  
    "Conectare la Telegram..."  
);
```

```
if (!client.connect(  
    host,  
    443  
)) {
```

```
    Serial.println(  
        "Eroare conectare Telegram!"  
    );
```

```
    return false;  
}
```

```
// =====  
// MULTIPART BOUNDARY  
// =====
```

```
String boundary =  
    "----ESP32CAMBoundary";
```

```
// =====  
// HEADER MULTIPART  
// =====
```

```
String head =  
    "--" + boundary + "\r\n"
```

```
    "Content-Disposition: form-data; "
```

"name=\"chat_id\"\\r\\n\\r\\n"

+ String(CHAT_ID) +

"\\r\\n"

"--" + boundary + "\\r\\n"

"Content-Disposition: form-data; "

"name=\"caption\"\\r\\n\\r\\n"

"ALERTA DE MISCARE - Camera X\\r\\n"

"--" + boundary + "\\r\\n"

"Content-Disposition: form-data; "

"name=\"photo\"; "

"filename=\"cameraX.jpg\"\\r\\n"

"Content-Type: image/jpeg\\r\\n\\r\\n";

```
// =====  
// FINAL MULTIPART  
// =====
```

String tail =
 "\\r\\n--" +
 boundary +
 "--\\r\\n";

```
// =====  
// URL TELEGRAM  
// =====
```

String url =
 "/bot" +

```
String(BOT_TOKEN) +  
"/sendPhoto";
```

```
// =====  
// LUNGIME CONTINUT  
// =====
```

```
uint32_t contentLength =  
    head.length() +  
    fb->len +  
    tail.length();
```

```
// =====  
// HTTP POST  
// =====
```

```
client.println(  
    "POST " +  
    url +  
    " HTTP/1.1"  
);
```

```
client.println(  
    "Host: " +  
    String(host)  
);
```

```
client.println(  
    "Content-Type: multipart/form-data; boundary=" +  
    boundary  
);
```

```
client.println(  
    "Content-Length: " +  
    contentLength  
);
```

```
    "Content-Length: " +  
    String(contentLength)  
);
```

```
client.println(  
    "Connection: close"  
);
```

```
client.println();
```

```
// =====  
// HEADER  
// =====
```

```
client.print(head);
```

```
// =====  
// FOTOGRAFIA  
// =====
```

```
client.write(  
    fb->buf,  
    fb->len  
);
```

```
// =====  
// FINAL  
// =====
```

```
client.print(tail);
```

```
// =====
```

```
// CITIRE RASPUNS TELEGRAM
// =====

unsigned long timeout =
    millis();

bool success = false;

while (
    client.connected() &&
    millis() - timeout < 10000
) {

    while (client.available()) {

        String line =
            client.readStringUntil('\n');

        Serial.println(line);

        if (
            line.startsWith(
                "HTTP/1.1 200"
            )
        ) {

            success = true;
        }

        timeout = millis();
    }
}

client.stop();
```

```
    return success;
}
```

```
// =====
// CAPTURA SI TRIMITERE TELEGRAM
// =====
```

```
void captureAndSend() {
```

```
    Serial.println();
    Serial.println(
        "Capturare fotografie..."
    );
```

```
    camera_fb_t* fb =
        esp_camera_fb_get();
```

```
    if (!fb) {
```

```
        Serial.println(
            "Eroare la capturarea fotografiei!"
        );
```

```
        return;
    }
```

```
    Serial.print(
        "Dimensiune fotografie: "
    );
```

```
    Serial.print(
        fb->len
```

```
);
```

```
Serial.println(  
  " bytes"  
);
```

```
bool result =  
  sendPhotoTelegram(fb);
```

```
if (result) {
```

```
  Serial.println(  
    "Fotografia a fost trimisa pe Telegram!"  
  );
```

```
} else {
```

```
  Serial.println(  
    "Fotografia NU a fost trimisa!"  
  );  
}
```

```
  esp_camera_fb_return(fb);  
}
```

```
// =====  
// SETUP  
// =====
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
Serial.println();
```

```
Serial.println(  
    "===== "  
);
```

```
Serial.println(  
    " SMART SECURITY CAMERA "  
);
```

```
Serial.println(  
    " CAMERA X "  
);
```

```
Serial.println(  
    "===== "  
);
```

```
// =====  
// PIR  
// =====
```

```
pinMode(  
    PIR_PIN,  
    INPUT  
);
```

```
// =====  
// CAMERA  
// =====
```

```
if (!initCamera()) {
```

```
    Serial.println(  
        "===== "  
    );  
    Serial.println(  
        " SMART SECURITY CAMERA "  
    );  
    Serial.println(  
        " CAMERA X "  
    );  
    Serial.println(  
        "===== "  
    );  
}
```



```
    "Camera nu a putut fi initializata!"  
  );
```

```
  while (true) {  
  
    delay(1000);  
  }  
}
```

```
// =====  
// WIFI  
// =====
```

```
connectWiFi();
```

```
// =====  
// WEB SERVER  
// =====
```

```
server.on(  
  "/",  
  HTTP_GET,  
  handleRoot  
);
```

```
server.on(  
  "/stream",  
  HTTP_GET,  
  handleStream  
);
```

```
server.begin();
```

```
Serial.println(  
    "Server web pornit!"  
);
```

```
Serial.print(  
    "Deschide in browser: http://"
```

```
);  
  
Serial.println(  
    WiFi.localIP()  
);
```

```
// =====  
// STABILIZARE PIR  
// =====
```

```
Serial.println(  
    "Asteptam stabilizarea PIR..."  
);
```

```
delay(5000);
```

```
Serial.println(  
    "SISTEM PREGATIT!"  
);  
}
```

```
// =====  
// LOOP  
// =====
```

```
void loop() {
```

```

// =====
// SERVER WEB
// =====

server.handleClient();

// =====
// PIR
// =====

int pirState =
  digitalRead(PIR_PIN);

if (pirState == HIGH) {

  unsigned long currentTime =
    millis();

  // =====
  // DETECTARE NOUA MISCARE
  // =====

  if (
    !motionActive &&
    currentTime - lastMotionTime >
    MOTION_COOLDOWN
  ) {

    motionActive = true;

    lastMotionTime =
      currentTime;

```

```
Serial.println();
```

```
Serial.println(  
  "=====  
);
```

```
Serial.println(  
  "🚨 MISCARE DETECTATA!"  
);
```

```
Serial.println(  
  "Camera X"  
);
```

```
Serial.println(  
  "=====  
);
```

```
  captureAndSend();  
}
```

```
} else {
```

```
  motionActive = false;  
}
```

```
  delay(50);  
}
```

VIII. CONCLUZII:

Prin realizarea acestui proiect a fost dezvoltat un sistem de monitorizare video bazat pe module ESP32-CAM și senzori PIR. Sistemul permite transmiterea imaginilor video în timp real și supravegherea simultană a mai multor zone prin intermediul unei interfețe web.

Utilizarea senzorilor PIR permite detectarea automată a mișcării, iar la apariția acesteia sistemul realizează o fotografie și o transmite prin intermediul unui bot Telegram. În acest mod, utilizatorul poate primi notificări și imagini de la distanță, fără a fi necesară supravegherea permanentă a interfeței web.

Interfața web centralizează fluxurile video ale celor patru camere, oferind o modalitate simplă de monitorizare a întregului sistem. Proiectul demonstrează integrarea modulelor ESP32-CAM, a senzorilor de mișcare, a comunicației Wi-Fi și a serviciului Telegram într-o singură aplicație.

În urma implementării, au fost urmărite atât partea hardware, prin realizarea conexiunilor dintre componente, cât și partea software, prin programarea camerelor și dezvoltarea interfeței de monitorizare.

IX. REFERINTE:

- ✓ Espressif Systems, ESP32 Series Datasheet.
- ✓ Espressif Systems, ESP32-CAM / ESP32 Camera Documentation.
- ✓ Arduino, Arduino Documentation – Digital I/O and GPIO.
- ✓ HC-SR501, PIR Motion Sensor – Technical Documentation.
- ✓ Telegram, Telegram Bot API Documentation.
- ✓ HTML, HTML Living Standard.
- ✓ MDN Web Docs, HTML, CSS și JavaScript Documentation.
- ✓ Python Software Foundation, Python Documentation – HTTP Server.
- ✓ Wi-Fi Alliance, Wi-Fi Technology Overview.
- ✓ <https://www.youtube.com/watch?v=mdaM53qIYgk>